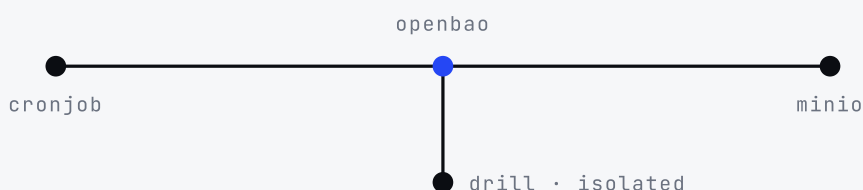


RAFT SNAPSHOTS OPENBAO

A CronJob with its own identity, verified archives to MinIO, and a restore you have actually rehearsed.



Raft Snapshots from the Cluster

A CronJob that logs in with its ServiceAccount, verifies the snapshot and writes it to S3 – and a restore drill that shows why two sets of unseal keys are needed

Thomas Zachmann

17 September 2026

Contents

What this is about

- Why by hand, and why without AI

- License and liability

- A word about the values in this guide

- Who this is for

- The building blocks

- The architecture in one picture

Part I – The target: MinIO

- Deployment

- Lifecycle instead of a cleanup script

Part II – The snapshot job

- Who may: policy and auth role

- The job

- The first run – and the silent error

- The staleness check

Part III – The restore drill

- Why isolated

- The instance

- The part everybody gets wrong

- The drill, self-contained

- The drill with the real snapshot

- What the drill does not prove

Part IV – What went wrong, and why

Part V – Operation

- Checking that it runs

- Keys apart from snapshots

- What is still open

Appendix A – All commands

Appendix B – Glossary

About the author

What this is about

Nº 1 ended with an uncomfortable sentence: the PVC is the only copy of the Raft data. Longhorn replicates the volume across nodes – that protects against a dead disk, not against a `helm uninstall` with a deleted PVC, a broken upgrade or a slip with `bao delete`. Everything in this OpenBao – KV secrets, the database roles from Nº 2, the CA from Nº 5 – hangs on that one volume.

This guide builds the backup: a CronJob that logs in to OpenBao with its ServiceAccount, runs `bao operator raft snapshot save`, verifies the archive and uploads it to MinIO. A second CronJob that raises the alarm when the newest snapshot is older than 26 hours. And – the most important part – a restore drill on an isolated second instance that makes visible the point everybody gets wrong the first time: a restore onto a fresh cluster needs **two** sets of unseal keys, one after the other.

It is the seventh part of a series. Nº 3 describes the same on a VM with a systemd timer and NFS; verification and restore procedure are taken from there. The cluster route is better in one respect: there is no periodic token in a file – the job logs in anew on every run.

Everything was carried out on the cluster. Four errors are in it, one of them silent.

Why by hand, and why without AI

“Write me a CronJob that backs up OpenBao to S3” delivers a working manifest in seconds. It does not deliver: why the job reported success although its retention never worked; why the `mc` image has no `grep` and what that does to `set -e`; why `kubectl cp` fails in that container; and why a restore without `-force` is rejected and with `-force` leaves the instance sealed.

Tools like [nyrvex](#), which generates the configuration of an AI platform’s secret store and identity provider, take these steps off your hands later. You should have walked them yourself once, to be able to judge what was generated.

The yardstick: whoever has worked through this guide can draw, on a blank sheet, the four steps of a restore onto a new cluster, which key set applies in which step, and why the drill instance must have no network.

License and liability

This guide is licensed under CC BY 4.0: it may be copied, shared and adapted, commercially too, as long as the author is credited. It is provided as is, without warranty. Everything in it was carried out in a development environment; whoever reproduces it elsewhere does so at their own risk.

A word about the values in this guide

MinIO credentials (`minioadmin123`, `openbao-snapshot-123`) are development values and printed on purpose. Unseal keys and root tokens appear nowhere – not even those of the drill instance; they lived only in shell variables and vanished with the shell.

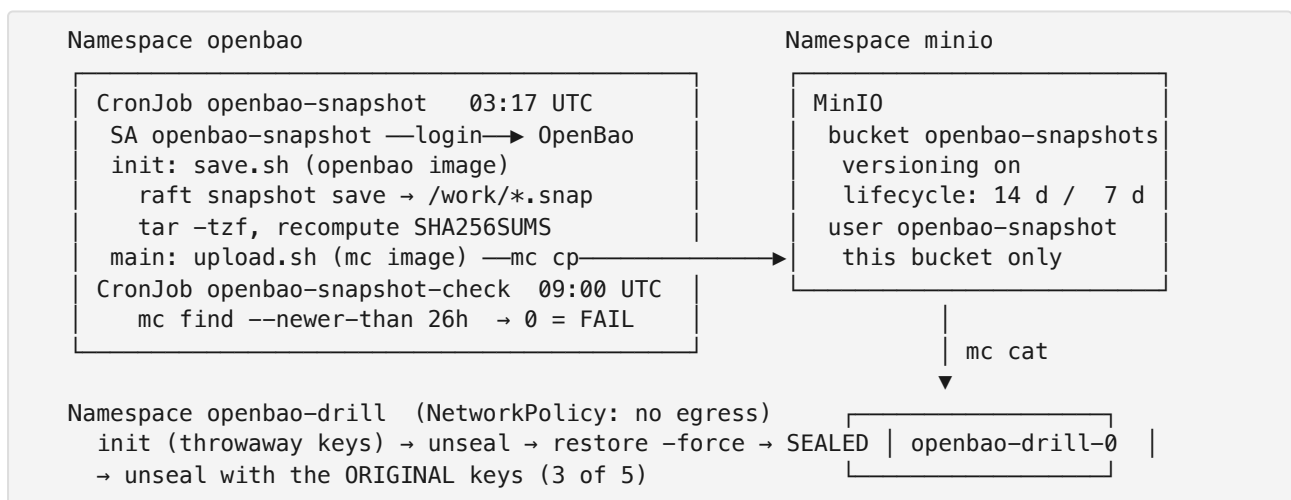
Who this is for

Readers who know N° 1 and what a CronJob is. N° 3 helps but is not required – the restore logic is explained again here. OpenBao basics are in my book **Vault in Practice** (Leanpub, leanpub.com/vault-in-practice).

The building blocks

Component	Version	Role
OpenBao in the cluster (N° 1)	2.6.2	the source; Kubernetes auth with role snapshot
MinIO (chart minio/minio)	5.4.0, RELEASE.2024-12-18	S3 target, bucket with versioning and lifecycle
mc	RELEASE.2024-11-21	upload in the job, lifecycle rule, staleness check
OpenBao image	2.6.2 (Alpine)	snapshot and verification in the job – has tar, sha256sum, wget
Second OpenBao instance	chart 0.29.4	restore drill, namespace openbao-drill, no egress

The architecture in one picture



Three things to take from the picture:

1. **Two containers, one task.** The OpenBao image can do bao and the verification, the mc image can do S3. An init container saves, the main container uploads; if the first fails, the second never runs.
2. **Retention belongs to the bucket, not the job.** A lifecycle rule deletes after 14 days, old versions after 7. The job knows nothing of it – and therefore cannot break it.

3. **The drill instance is a prisoner.** A restored OpenBao holds every lease of the original. If it reaches the database, it can drop real users. Hence the NetworkPolicy before the first snapshot goes in.

Part I – The target: MinIO

Deployment

A standalone MinIO with one PVC is enough as a target in the same cluster – that is no offsite copy, but it separates the snapshots from the OpenBao's PVC. Whoever has Longhorn backups to an external target gets the second tier with it; an external S3 would be a URL change in the Secret.

```
mode: standalone
persistence:
  size: 20Gi
  storageClass: longhorn
rootUser: minioadmin
rootPassword: minioadmin123

buckets:
- name: openbao-snapshots
  versioning: true          # an overwritten snapshot stays recoverable
policies:
- name: openbao-snapshots-rw
  statements:
  - resources: ["arn:aws:s3:::openbao-snapshots"]
    actions: ["s3:ListBucket", "s3:GetBucketLocation"]
  - resources: ["arn:aws:s3:::openbao-snapshots/*"]
    actions: ["s3:PutObject", "s3:GetObject", "s3:DeleteObject"]
users:
- accessKey: openbao-snapshot
  secretKey: openbao-snapshot-123
  policy: openbao-snapshots-rw
```

```
helm repo add minio https://charts.min.io/
helm upgrade --install minio minio/minio --version 5.4.0 \
  -n minio --create-namespace -f k8s/minio-values.yaml
```

The chart creates bucket, policy and user through a post-install job. Verify, with the chart's own mc image:

```
$ kubectl run -n minio --rm -i --restart=Never --image=quay.io/minio/mc:RELEASE.2024-11-21T17-21-54Z mc --command -- sh -c '
  mc alias set local http://minio:9000 minioadmin minioadmin123
  mc version info local/openbao-snapshots
  mc admin user list local
  mc alias set snap http://minio:9000 openbao-snapshot openbao-snapshot-123
  echo test | mc pipe snap/openbao-snapshots/probe.txt && mc rm snap/openbao-snapshots/p
robe.txt
  mc mb snap/other'
local/openbao-snapshots versioning is enabled
enabled    openbao-snapshot    openbao-snapshots-rw
Created delete marker `snap/openbao-snapshots/probe.txt` ...
mc: <ERROR> Unable to make bucket `snap/other`. Access Denied.
```

The snapshot user can write into its bucket and nothing else. The Access Denied is the test that counts.

Two small things that cost time: the image is `quay.io/minio/mc`, not `minio/mc` on Docker Hub – the tag no longer exists there, and `kubectl run` then hangs in `ImagePullBackOff`. And `mc` warns on every call that commands including credentials end up in the container log – which is why the job below has the credentials in the environment (`MC_HOST_s3`), not on the command line.

Lifecycle instead of a cleanup script

```
$ mc ilm rule add --expire-days 14 --noncurrent-expire-days 7 local/openbao-snapshots
Lifecycle configuration rule added with ID `daltkkk79e2s76cvv84g`
```

Current objects expire after 14 days, older versions after 7. Why this has to be server-side and not in the job is shown in Part II.

Part II – The snapshot job

Who may: policy and auth role

```
/ $ bao policy write snapshot - <<'EOF'
path "sys/storage/raft/snapshot" { capabilities = ["read"] }
EOF
/ $ bao write auth/kubernetes/role/snapshot \
    bound_service_account_names=openbao-snapshot \
    bound_service_account_namespaces=openbao \
    token_policies=snapshot token_ttl=15m
```

One line of policy, one path, read. The token lives 15 minutes – no snapshot needs longer. That is the difference from the VM (Nº 3): there a periodic token sits in `/etc/openbao/snapshot.token`, because a systemd timer has no identity. A pod has one.

The job

```
apiVersion: batch/v1
kind: CronJob
metadata:
  name: openbao-snapshot
  namespace: openbao
spec:
  schedule: "17 3 * * *"
  concurrencyPolicy: Forbid
  jobTemplate:
    spec:
      backoffLimit: 1
      template:
        spec:
          serviceAccountName: openbao-snapshot
          restartPolicy: Never
          volumes:
            - name: work
              emptyDir: {}
            - name: scripts
              configMap:
                name: openbao-snapshot-scripts
                defaultMode: 0755
          initContainers:
            - name: save
              image: quay.io/openbao/openbao:2.6.2
              command: ["/scripts/save.sh"]
              volumeMounts:
                - { name: work, mountPath: /work }
                - { name: scripts, mountPath: /scripts }
          containers:
            - name: upload
              image: quay.io/minio/mc:RELEASE.2024-11-21T17-21-54Z
              command: ["/scripts/upload.sh"]
              envFrom:
                - secretRef:
                    name: openbao-snapshot-s3
              volumeMounts:
                - { name: work, mountPath: /work }
                - { name: scripts, mountPath: /scripts }
```

save.sh – login, snapshot, verification:

```
#!/bin/sh
set -eu
export BAO_ADDR=http://openbao.openbao.svc:8200
JWT=$(cat /var/run/secrets/kubernetes.io/serviceaccount/token)
BAO_TOKEN=$(bao write -field=token auth/kubernetes/login role=snapshot jwt="$JWT")
export BAO_TOKEN
TS=$(date -u +%Y%m%dT%H%M%SZ)
OUT=/work/openbao-$TS.snap
bao operator raft snapshot save "$OUT"
# A Raft snapshot is a gzipped tar with meta.json, state.bin and
# SHA256SUMS. Recompute - OpenBao has no 'snapshot inspect'.
tar -tzf "$OUT" >/dev/null
tar -xz0f "$OUT" SHA256SUMS > /work/expected
for f in meta.json state.bin; do
    have=$(tar -xz0f "$OUT" "$f" | sha256sum | cut -d' ' -f1)
    want=$(grep " $f\$" /work/expected | cut -d' ' -f1)
    [ "$have" = "$want" ] || { echo "integrity check failed for $f"; rm -f "$OUT"; exit 1; }
done
echo "snapshot ok: $OUT ($(wc -c < "$OUT") bytes)"
bao token revoke -self >/dev/null 2>&1 || true
```

upload.sh – upload only:

```
#!/bin/sh
set -eu
f=$(ls /work/openbao-*.snap)
mc cp "$f" s3/openbao-snapshots/
echo "uploaded $(basename "$f"); bucket now:"
mc ls s3/openbao-snapshots/
```

The MinIO credentials come from a Secret as MC_HOST_s3 – a URL with embedded credentials that mc understands as alias s3. For the demo the Secret is a manifest; in a real setup it would come from the KV engine via ExternalSecret (Nº 4).

The first run – and the silent error

```
$ kubectl create job -n openbao --from=cronjob/openbao-snapshot snapshot-manual-1
$ kubectl get job -n openbao snapshot-manual-1
NAME                STATUS      COMPLETIONS   DURATION
snapshot-manual-1   Complete   1/1           19s

$ kubectl logs -n openbao job/snapshot-manual-1 -c save
snapshot ok: /work/openbao-20260917T121637Z.snap (64540 bytes)

$ kubectl logs -n openbao job/snapshot-manual-1 -c upload
`/work/openbao-20260917T121637Z.snap` -> `s3/openbao-snapshots/openbao-20260917T121637Z.sn
ap`
/scripts/upload.sh: line 6: grep: command not found
uploaded openbao-20260917T121637Z.snap; bucket now:
[2026-09-17 12:16:39 UTC] 63KiB STANDARD openbao-20260917T121637Z.snap
```

The job is Complete, the snapshot is in the bucket – and in the middle it says grep: command not found. The first version of upload.sh had a retention pipeline (mc ls --json | grep | cut | sort | head | while ... mc rm). The mc image has no grep, no sed, no awk. And set -e did not help: in a

pipeline only the exit code of the **last** command counts, and head was happy.

A backup job that reports success while part of its work never happens is the most dangerous error in this guide. It would have surfaced only once the bucket was full. Two consequences:

- Retention out of the job, into the lifecycle rule (Part I). What the job does not do, it cannot do silently wrong.
- A job script may only contain commands that exist in the image. Check beforehand: `for t in grep sed awk cut sort head; do command -v $t || echo "$t MISSING"; done`.

The staleness check

The job itself cannot report that it did not run. A second CronJob checks the **result** – not the timer:

```
apiVersion: batch/v1
kind: CronJob
metadata:
  name: openbao-snapshot-check
  namespace: openbao
spec:
  schedule: "0 9 * * *"
  jobTemplate:
    spec:
      backoffLimit: 0
      template:
        spec:
          restartPolicy: Never
          containers:
            - name: check
              image: quay.io/minio/mc:RELEASE.2024-11-21T17-21-54Z
              envFrom:
                - secretRef:
                    name: openbao-snapshot-s3
              command:
                - sh
                - -c
                - |
                  set -eu
                  n=$(mc find s3/openbao-snapshots --name "openbao-*.snap" --newer-than 26
h | wc -l)
                  echo "snapshots younger than 26h: $n"
                  [ "$n" -gt 0 ] || { echo "STALE: no recent snapshot"; exit 1; }
```

```
$ kubectl logs -n openbao job/check-manual-1
snapshots younger than 26h: 1
```

And the counter-test with `--newer-than 1s`: `count=0`, STALE, exit 1. A failed job is visible in `kubectl get jobs` and – if there is monitoring – as `kube_job_failed` in Prometheus. There is none here; that is the open item in Part V.

Part III – The restore drill

Why isolated

A restored OpenBao is not empty. It holds every lease of the original: the dynamic database users from N° 2, the tokens, the certificates. If it runs unsealed and reaches the database, it executes `DROP ROLE` when a lease expires – against the **real** database, for a user the original is still using. Hence, before the first snapshot:

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: drill-isolation
  namespace: openbao-drill
spec:
  podSelector: {}
  policyTypes: [Ingress, Egress]
  ingress:
    - from:
      - podSelector: {}
  egress:
    - to:
      - namespaceSelector:
          matchLabels:
            kubernetes.io/metadata.name: kube-system
  ports:
    - port: 53
      protocol: UDP
```

No egress except DNS. `kubectl exec` needs no network path into the pod; it goes through the kubelet. The proof:

```
$ kubectl exec -n openbao-drill openbao-drill-0 -- wget -q0- -T 3 http://openbao.openbao.svc:8200/v1/sys/health
wget: download timed out
```

The instance

A second Helm release with the same values as N° 1, minus ingress and UI, 2 Gi PVC:

```
helm upgrade --install openbao-drill openbao/openbao --version 0.29.4 \
  -n openbao-drill -f k8s/drill-values.yaml
```

The part everybody gets wrong

A restore onto a **new** cluster needs two sets of unseal keys:

1. New instance: init -> NEW keys, NEW root token
2. Unseal with the NEW keys -> running, but empty
3. restore -force -> the snapshot's keyring replaces the new one
4. Unseal with the ORIGINAL keys -> the new ones are worthless

-force is required because OpenBao checks whether the snapshot matches its own keyring. It does not – different cluster, different keys – and without the flag it refuses for exactly that reason. The keys from step 1 are throwaway material for the minutes between step 2 and 3.

The drill, self-contained

To prove the procedure without needing the production keys: the drill instance first plays the original (A), then the new cluster (B).

```
== A: init (1 share), unseal, write a marker, snapshot
$ bao operator init -key-shares=1 -key-threshold=1
$ bao operator unseal <key A>
$ bao secrets enable -path=secret kv-v2
$ bao kv put secret/drill/marker value=written-before-snapshot
$ bao operator raft snapshot save /tmp/drillA.snap      # 19996 bytes

== delete the data, new pod -> uninitialised
$ rm -rf /openbao/data/*; kubectl delete pod openbao-drill-0
Initialized    false

== B: init -> NEW keys
$ bao operator init -key-shares=1 -key-threshold=1
$ bao operator unseal <key B>
Sealed         false

== restore WITHOUT -force:
$ bao operator raft snapshot restore /tmp/drillA.snap
* could not verify hash file, possibly the snapshot is using a different set
  of unseal keys; use the snapshot-force API to bypass this check

== restore WITH -force:
$ bao operator raft snapshot restore -force /tmp/drillA.snap
$ bao status | grep Sealed
Sealed         true

== unseal with key B:
Code: 400      <- rejected

== unseal with key A:
Sealed         false

== marker there? (with root token A)
$ bao kv get -field=value secret/drill/marker
written-before-snapshot

== root token B?
* permission denied
```

Every line is evidence: without `-force`, rejection with a clear message. With `-force`, sealed. The new key is rejected, the old one unseals. The data is the snapshot's, including the root token. The intermediate instance's root token is dead.

A detail on the side: `bao operator raft snapshot restore` prints `Error properly closing policy file: ... file already closed at the end`. That is a cosmetic CLI error, not a problem with the restore – `bao status` afterwards is the truth.

The drill with the real snapshot

Then the same procedure with the production snapshot from MinIO:

```
$ kubectl run -n minio --restart=Never --image=quay.io/minio/mc:... mcbox --command -- sleep 600
$ kubectl exec -n minio mcbox -- mc cat s3/openbao-snapshots/openbao-20260917T121637Z.snap > prod.snap
$ tar -tzf prod.snap
meta.json state.bin SHA256SUMS SHA256SUMS.sealed
$ tar -xzOf prod.snap state.bin | sha256sum          # matches SHA256SUMS

$ kubectl cp prod.snap openbao-drill/openbao-drill-0:/tmp/prod.snap
(drill: rm -rf data, init, unseal with its own key)
$ bao operator raft snapshot restore -force /tmp/prod.snap
$ bao status | grep -E 'Sealed|Total|Threshold'
Sealed          true
Total Shares    5
Threshold       3
```

Total Shares 5, Threshold 3 – that is the production OpenBao's keyring, not the drill instance's 1/1. From here it takes three of the five real unseal keys, and those belong in a terminal, not in a guide. The instance stays sealed; unsealed, it shows all secrets from N° 2, 4 and 5 – safe to look at, because the NetworkPolicy holds.

Two tool traps on the way, both real:

- `kubectl run --rm ... > file` mixes the message `pod "..."` deleted into the file. For binary data: start the pod without `--rm`, redirect `kubectl exec ... mc cat`, delete the pod afterwards.
- `kubectl cp` needs `tar` **inside the container**. The `mc` image has none; `kubectl cp` then fails silently or with `tar: not found`. The route via `exec` and `stdout` is more robust.

What the drill does not prove

That the three keys work. Only whoever has them can – and should do it once while the drill instance stands. Afterwards:

```
kubectl delete namespace openbao-drill
```

Part IV – What went wrong, and why

Error	Symptom	Cause	Fix
Silent retention	job Complete, log says grep: command not found	mc image without grep/sed/awk; pipeline exit code is head's	retention as lifecycle rule; scripts only with commands from the image
Image not found	kubectl run hangs	minio/mc on Docker Hub no longer has the tag	quay.io/minio/mc, tag from the chart
Restore refused	could not verify hash file ... different set of unseal keys	snapshot comes from another keyring	-force - and knowing that the original keys apply afterwards
Corrupt snapshot file	Unrecognized archive format	kubectl run --rm appended "pod deleted"	kubectl exec ... mc cat instead of run --rm
kubectl cp fails	empty file	no tar in the mc container	fetch data via stdout

And one that happened to me and is in no manifest: the drill instance's keys lived in shell variables that were lost between two calls – the restore was done, but the key to unseal was gone. For a throwaway instance irrelevant; for the real one it would be the end. Unseal keys belong in the password manager **before** you use them.

Part V – Operation

Checking that it runs

```
kubectrl get cronjob -n openbao          # LAST SCHEDULE
kubectrl get jobs -n openbao             # Complete / Failed
kubectrl logs -n openbao job/<name> -c save
kubectrl logs -n openbao job/<name> -c upload
```

The check job is the one to watch: a Failed there means no snapshot for 26 hours.

Keys apart from snapshots

The snapshots are encrypted with the master key that the unseal keys reconstruct. If the keys sit in the same MinIO, the same cluster, on the same host as the snapshots, one loss takes both. The keys belong in the password manager; MinIO belongs, when it gets serious, on another device or in an external S3.

What is still open

- **No alert.** A failed check job is in `kubectrl get jobs`, nothing more. With Prometheus/kube-state-metrics: `kube_job_failed` on `openbao-snapshot-check`. Without: an `mc` call in the check that sends a message.
- **MinIO in the same cluster.** A cluster loss takes the snapshots with it. The next step is bucket replication to an external target (`mc replicate`) or an external S3 as the destination.
- **The real drill.** Until somebody has entered the three keys into the drill instance, the production restore is an assumption.
- **No audit device** (N° 1). Who took a snapshot when is only in the job logs.

Appendix A – All commands

```
# — MinIO —————
helm upgrade --install minio minio/minio --version 5.4.0 -n minio --create-namespace -f k8s/minio-values.yaml
mc ilm rule add --expire-days 14 --noncurrent-expire-days 7 local/openbao-snapshots

# — OpenBao —————
bao policy write snapshot - <<'EOF'
path "sys/storage/raft/snapshot" { capabilities = ["read"] }
EOF
bao write auth/kubernetes/role/snapshot bound_service_account_names=openbao-snapshot \
    bound_service_account_namespaces=openbao token_policies=snapshot token_ttl=15m

# — Jobs —————
kubectl apply -f k8s/snapshot-cronjob.yaml -f k8s/snapshot-check-cronjob.yaml
kubectl create job -n openbao --from=cronjob/openbao-snapshot snapshot-manual-1
kubectl logs -n openbao job/snapshot-manual-1 -c save
kubectl logs -n openbao job/snapshot-manual-1 -c upload
kubectl create job -n openbao --from=cronjob/openbao-snapshot-check check-manual-1

# — Restore drill —————
kubectl create ns openbao-drill && kubectl apply -f k8s/drill-networkpolicy.yaml
helm upgrade --install openbao-drill openbao/openbao --version 0.29.4 -n openbao-drill -f k8s/drill-values.yaml
kubectl exec -n minio mcbox -- mc cat s3/openbao-snapshots/<file> > prod.snap
kubectl cp prod.snap openbao-drill/openbao-drill-0:/tmp/prod.snap
# inside the drill pod:
bao operator init -key-shares=1 -key-threshold=1          # throwaway keys
bao operator unseal <throwaway-key>
bao operator raft snapshot restore -force /tmp/prod.snap
bao operator unseal                                       # 3x ORIGINAL keys
kubectl delete namespace openbao-drill                   # afterwards
```

Appendix B – Glossary

Raft snapshot – Consistent dump of the Raft storage: gzipped tar with `meta.json`, `state.bin`, `SHA256SUMS` (and `SHA256SUMS.sealed`). Encrypted with the master key of the cluster it came from.

Master key / unseal keys – The master key encrypts the storage; Shamir splits it into unseal keys. A snapshot brings its keyring with it.

-force – Bypasses the check whether the snapshot matches the current keyring. Needed when restoring onto another cluster; afterwards the snapshot's keys apply.

Init container – Runs before the main containers; if it fails, the pod does not continue. Here: snapshot and verification.

Lifecycle rule – Server-side retention in S3/MinIO: objects and old versions expire after days.

MC_HOST_<alias> – Environment variable by which mc knows an alias with credentials, without having them on the command line.

Staleness check – Checking the result (newest backup) instead of the trigger (timer ran).

NetworkPolicy – Kubernetes object restricting a pod's ingress/egress. Here: no egress except DNS for the drill instance.

Lease – Lifetime of a dynamic secret. A restored instance holds all leases of the original and would let them expire.

About the author

Thomas Zachmann is a freelance platform engineer based in Hamburg. He builds enterprise platforms for Kubernetes, cloud and AI workloads – from identity and secrets through CI/CD and GitOps to observability – so that the in-house team can run them without him afterwards. These Field Notes come out of that work. For project enquiries: thomaszachmann.de.

The job reported success. Its retention had never worked.

This guide builds the backup for the OpenBao from N° 1: a CronJob that logs in via ServiceAccount, takes the Raft snapshot, recomputes the archive's checksums and uploads to MinIO; a second one that raises the alarm when none arrived for 26 hours. Retention as a lifecycle rule on the bucket – not in the job, for good reason.

And a restore drill on an isolated second instance that shows the point everybody gets wrong the first time: `restore -force` leaves the instance sealed, the new key is rejected, only the original key unseals. With the real snapshot up to Total Shares 5.

```
-- Four steps, two key sets
bao operator init                # throwaway keys
bao operator unseal <new key>
bao operator raft snapshot restore -force prod.snap
bao operator unseal <original-key> x3 # the new ones are worthless
```



openbao	2.6.2 · raft · SA login
minio	chart 5.4.0 · versioning · ilm 14d/7d
mc	RELEASE.2024-11-21 · no grep
drill	openbao-drill · NetworkPolicy
policy	1 line · read

21 pages · 5 parts · 2 appendices
Deep dive: **Vault in Practice**
leanpub.com/vault-in-practice
Set in Inter Tight, Inter and JetBrains Mono.
All output is real terminal output.
CC BY 4.0 · share freely · no warranty